

# DATA SECURITY AND PRIVACY

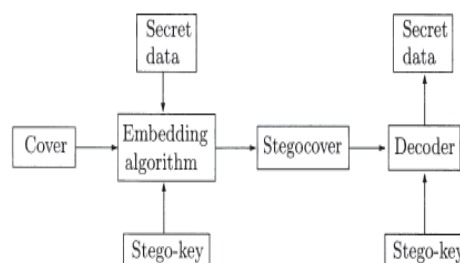
|                     |                    |                        |                 |
|---------------------|--------------------|------------------------|-----------------|
| Course Code         | BAD703             | Semester               | 7 <sup>th</sup> |
| CIE Marks           | 50                 | Total Marks            | 100             |
| Teaching Hours/Week | (L:T:P: S) 4:0:0:0 | Total Credits          | 04              |
| SEE Marks           | 50                 | Exam Hours             | 3               |
| Hours of Pedagogy   | 50                 | Examination type (SEE) | Theory          |

## Module 5

**Data Security: Data hiding in Text-Basic features, Applications of data hiding, Watermarking, Intuitive Methods, Simple Digital methods, Data hiding in Text, Innocuous Text, Mimic Functions. Data hiding in Images: LSB encoding, BPCS Steganography,**

In today's digital world, all forms of data—text, images, audio, and video—can be easily digitized, stored, and transmitted. However, this convenience also leads to concerns such as illegal access, tampering, and copyright violations. To address these issues, steganography—the art and science of hiding data—is used to embed secret information within digital media without altering its visible properties. Unlike cryptography, which transforms data into unreadable formats to secure it, steganography conceals the very existence of the hidden message. The modified file, called a stego object, looks and behaves like the original cover file (coverttext, coverimage, coveraudio, or covervideo), making detection difficult. Steganography can even be layered with encryption for additional security and is preferred in situations where raising suspicion could have serious consequences.

Despite its advantages, steganography faces significant technical challenges, particularly with media that undergo lossy compression, which eliminates redundancies—often the very spots used for hiding data. To overcome this, robust embedding methods are required. The typical steganographic process includes an encoding algorithm that takes secret data, a cover file, and an optional key to generate the stego object. The decoding algorithm, using the stego object and optionally a key, then retrieves or verifies the embedded data. In some cases, the decoder only checks for the presence of embedded data, such as a digital watermark used to prove ownership. Organizations like Watermarking World support research and collaboration in digital watermarking, a key area within steganography.



The above figure shows the main steps in a typical steganographic method. The encoding algorithm receives three inputs, the secret data to be embedded, the cover data, and an optional steganographic key. The algorithm then produces a stego cover that can be stored and/or transmitted. The decoding algorithm receives the stegocover and the (optional) stego-

key, and extracts the secret data. In some algorithms, the decoder cannot actually extract the data and can only answer the question, "Are these data really embedded in the file being examined?" This makes sense in cases where the hidden data are a watermark, originally placed in the cover to prove ownership or simply for pride of ownership. *Watermarking World* is an international meeting point for scientists, researchers, and organizations active in digital watermarking (one of the areas of steganography). It is the first nonprofit organization dedicated to digital watermarking. It provides means and services to the digital watermarking community. Its goal is to facilitate communication among those active in this field and to contribute to research and implementation efforts.

## BASIC FEATURES

This section highlights the key features of a data hiding algorithm.

**Embedding capacity**, also known as payload, refers to the amount of secret data that can be hidden within a larger file, known as the cover or carrier, which may be text, image, audio, or video. It is typically measured in bits-per-bit (bpb). While a small embedding capacity may seem limiting, such algorithms may offer better robustness and are ideal when only a small amount of data needs to be hidden.

**Invisibility**, also called perceptual transparency or algorithm quality, measures how much distortion is introduced to the cover after embedding the hidden data. Even a large capacity is useless if it significantly distorts the cover. Invisibility is qualitative and best evaluated by human observers comparing the original and modified files. If no noticeable difference is detected, the algorithm is considered highly invisible.

**Undetectability** refers to the algorithm's ability to resist detection through statistical analysis. For instance, in images, prediction errors in pixel values often follow a Laplace distribution. If the embedded data cause noticeable statistical anomalies, the file may raise suspicion. Thus, a good steganographic method must preserve the statistical properties of the cover, even though this is different from perceptual invisibility.

Pixels in images are not independent; nearby pixels usually share similar values. Hence, a pixel's value can often be predicted based on its neighbors, typically using a weighted average. The difference between this prediction and the actual value is the prediction error, which tends to follow a Laplace distribution. If hidden data alter this distribution, the steganography may become detectable.

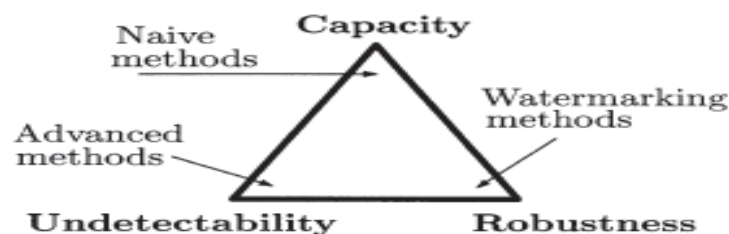
**Robustness** is another important feature, referring to the algorithm's ability to retain hidden data even after the cover has undergone transformations such as lossy compression, analog-to-digital conversion, or image processing operations like filtering, rotating, cropping, or sharpening. Robustness is especially critical when hiding copyright or ownership data (watermark), as such information must remain intact despite attempts to remove or damage it through these modifications.

**Tamper resistance** is the algorithm's ability to prevent unauthorized changes to the hidden data. An attacker may not only try to destroy the embedded data but also to replace them. Tamper-resistant algorithms make such alterations extremely difficult, especially for critical data such as copyright information, which must remain unaltered for decades—70 years after the author's death for text and 50 years for audio. Such data must withstand future technological developments. Experience in computing shows that enhancing one feature often compromises another. In steganography, there is typically a tradeoff between embedding capacity and robustness (and tamper resistance). The more robust a method, the less data it can embed. Also, secret data should always be embedded within the body of the cover, not in headers or footers, to survive format conversions that may strip metadata. However, it is also true that no method is foolproof—any system created by one person can often be defeated by another, especially a determined and knowledgeable attacker who may view the challenge as personal.

**Signal-to-Noise Ratio (SNR)** is commonly used to measure invisibility or, conversely, detectability. Unlike communication systems where a high SNR is desired, in steganography, the cover is treated as noise and the embedded data as the signal. Therefore, a low SNR is ideal, as it indicates that the hidden message is well concealed within the noise of the cover.

**Cover Escrow vs. Blind Cover:** Some steganographic methods require the **original cover** to extract the embedded data—these are said to use a **cover escrow** approach. Others can retrieve the hidden message without needing the original cover; such methods are called **blind** or **oblivious**.

Some of the key requirements in steganographic algorithms—such as embedding capacity, robustness, and undetectability—are often in conflict with one another. As a result, no single algorithm can fully satisfy all three simultaneously. Figure below illustrates the tradeoffs between these three features. It shows that **naive steganographic methods** may offer **high embedding capacity**, but typically suffer from **poor robustness and low undetectability**. On the other hand, **advanced algorithms** can achieve **high undetectability**, but usually provide **low embedding capacity** and **limited robustness**. Algorithms designed for **watermarking** tend to focus on **robustness**, yet they often result in **low embedding capacity** and may not be fully undetectable



## OTHER FEATURES:

Like any other discipline that uses computers, steganography has many other "digital" features, a few of which are listed here.

1. Much as encryption methods use an encryption key, steganographic algorithms can use a stego-key to increase security. Kerckhoffs' principle, discussed in the Introduction, applies to steganography as well. When applied to steganography, this principle, due to the Dutch

linguist Auguste Kerckhoffs von Nieuwenhoff, states that the security of a hidden message must depend on keeping the stego-key secret. It must not depend on keeping the hiding algorithm secret.

2. The complexity of the algorithm is also an important feature. A computationally intensive method may result in greater invisibility and robustness, and these may justify the extra time needed to embed and retrieve the hidden data. The execution time of an algorithm may often be less important than its other features. An example is a music CD with copyright information embedded. The copyright has to be embedded into the music just once, following which the digital information (music plus copyright) can be recorded on as many CDs as needed without any extra computations.

3. Asymmetric hiding of data may be a desirable feature. A slow encoding algorithm may many times be acceptable if the decoding algorithm is simple and fast.

4. The use of error-correcting codes is an advantage. Many types of processing may modify the cover and thus corrupt the hidden data. An ideal steganographic algorithm may be able to detect and even correct the embedded data after such a process.

5. Certain attacks on hidden data may destroy part of the cover. Ideally, it should be possible to retrieve that part of the embedded data that's hidden in the remaining part of the cover.

6. A parallel steganographic algorithm is one that can be executed by several processors simultaneously, each embedding (or retrieving) part of the data. Such an algorithm may be important to steganalysts trying to break a steganographic algorithm.

## APPLICATIONS OF DATA HIDING

- The most straightforward application of data hiding is to conceal private or sensitive information within a cover. The hidden data may be embedded either in its raw form or after being encrypted. Since the volume of data to be hidden could be substantial, it is practical to compress it before encryption and embedding. This method allows parties to exchange messages indirectly—without the need to send emails, use instant messaging, or log into specific accounts. A sender can simply post a cover file (such as a text, image, or audio file) on a public platform under a pseudonym known to the receiver. The receiver can then download the file and extract the secret message using a stego-key. For example, a text file with a hidden message may be uploaded to a newsgroup, or an image file could be posted on an online auction site by a sender posing as a seller.
- Another key application of data hiding is embedding a digital identifier or signature, known as a **watermark**, into the cover. A watermark typically contains a small amount of data that asserts ownership, authorship, or affiliation. In this context, the algorithm used may have low embedding capacity, but it must be highly resistant to tampering or removal. The watermark allows the owner to verify, “Is this file mine?” In some cases, the watermark is meant to be publicly verifiable to answer, “Whose file is this?” For instance, an artist displaying their work on a website may embed a watermark—partly visible to deter piracy, and partly hidden to remain intact even if the visible part is stripped away.

Watermarking is especially valuable in the **publishing and broadcasting industries**, where the advancement of multimedia technologies opens doors to both innovation and increased risk of unauthorized copying. **Fingerprinting**, a variation of watermarking, embeds a unique watermark in each distributed copy. This is useful when tracking illegal distribution—for example, identifying the source of a pirated music CD. Another variant is **traitor tracing**, in which every recipient of a sensitive document receives a uniquely fingerprinted copy. If the document is leaked, the individual responsible can be pinpointed with ease.

- A third application is **tamper-proofing**. Here, data is embedded in a way that reveals any modifications made to the cover. The purpose is to answer the question, “Has this file been altered?” In audio files, tamper-proofing should detect word deletions or changes, but ignore minor adjustments like left-right stereo balance. In images, it should identify object removal but not be affected by changes like gamma correction or color space adjustments. A special case of this is **file sealing**, where a checksum is calculated and embedded into the file, allowing the creator to later verify if the file’s integrity has been compromised.

Government restrictions on strong encryption have led to another important use of data hiding. When legal constraints prevent the use of robust encryption, steganography becomes a viable alternative for privacy protection. As lawmakers become more aware of steganography’s benefits, they may be more willing to lift ineffective restrictions on cryptography.

- Audio-specific applications of steganography also exist. For example, **caller ID information** can be embedded into telephone conversations to help receivers identify callers. A company contracting a radio station to air commercials could embed a **signature** within the ad. A computer system can monitor the broadcast, detect the signature, and report when and how often the commercial was aired—even identifying cases where it was cut short.
- Lastly, **feature location** or **feature tagging** is an important application in which metadata such as captions, annotations, names, and timestamps are embedded into the cover. This use case typically requires **high embedding capacity**, though it may not need the robustness that watermarking demands

## WATERMARKING

The term **watermarking** refers to techniques used to embed ownership or authorship information within important digital data. A watermark might be a signature placed for the creator’s pride—just as artists sign their paintings—or it may serve a protective purpose when digital content such as images, videos, audio, or even text is released to the public (e.g., via websites, magazines, museums, or collectors). Watermarks enable content creators to identify their work if it is later copied, stolen, or altered. For instance, creating a road map requires significant effort and resources, but once digitized, it can easily be duplicated and slightly modified for illegal resale. A hidden watermark in the map helps the original creator assert ownership. Importantly, the watermark should not significantly increase the size of the

data and must be **robust**—resisting damage from filtering, compression, cropping, and other transformations.

Watermarking can be implemented through **steganographic methods**, where a repetitive string (like the owner's name) is embedded in the original data and can later be retrieved using a secret key. However, there is a **fundamental difference** between **steganography** and **watermarking**. In steganography, the focus is on the **hidden message**, and the **cover** is merely a vehicle. High embedding capacity is essential, though the hidden data may be fragile and susceptible to compression or transformation. In contrast, in watermarking, the **cover** (e.g., an image) is the valuable asset, and the **hidden data** (e.g., a signature) is secondary. Here, embedding capacity is not critical, but robustness of the watermark is essential. Therefore, watermarking requires specialized steganographic techniques. A closely related concept is **fingerprinting**, which involves embedding a unique identifier or serial number in each distributed copy of digital data. For example, by embedding a distinct serial number in every sold copy of a software program, a company can trace pirated copies back to the original buyer. [Cox 02] is a general reference for watermarking.

## INTUITIVE METHODS

1. Write a message on a wooden tablet and cover it with a coat of wax on which another, innocuous message is written. This method is related by Greek historians. Modern variations include hiding a message in a hollow heel or in a false bottom of a suitcase.
2. Choose a messenger, shave his head, tattoo the message on his head, wait for the hair to grow, and send him to his destination, where his head is shaved again. This method is attributed to Histiaeus.
3. Use invisible ink, made of milk, vinegar, fruit juice, or even urine, and hide the message between the lines of a seemingly innocent letter. When the paper is heated up, the text of the hidden message slowly appears.
4. The letters constituting the data are concealed in the second letter of every word of a specially constructed *cover text*, or in the third letter of the first word of each sentence. An example is the data "cover blown" that can be hidden in the specially contrived cover text "Accepted you over Neil Brown. About ill Bob Ewing, encountered difficulties." A built-in computer dictionary can help in selecting the words, but such specially-constructed text often looks contrived and may raise suspicion. A variation on this method starts by writing the words of the secret message vertically. Each word becomes the first word of a line and the steganographer (writer or originator of the message) has to complete each line by adding more text. The following is an example of how the message "Come to our place at midnight" can be hidden in this way. Come see how the swallows fly to the south island helped by our fine weather. They locate the place in the immense ocean by navigating at night, following the clouds which form around midnight above the shore.
5. An ancient method to hide data uses a large piece of text where small dots are placed under the letters that are to be hidden. For example, this paragraph has line placed under certain letters that together spell the message "i am hidden." A variation of this method slightly perturbs certain letters from their original positions to indicate the hidden data.
6. A series of lists of alternative phrases from which a paragraph can be built can be used, where each choice of a phrase from a list conceals one letter of a message. This method was

published in 1518 in *Polygraphiae* by Johannes Trithemius, and was still used during World War II.

7. Check digits. This idea is used mostly to verify the validity of important data, such as bank accounts and credit card numbers, but can also be considered a method for hiding validation information in a number. A simple example is the check digit used in the well-known *international standard book number* (ISBN), assigned to every book published. This number has four parts, a country code, a publisher code, a book number assigned by the publisher, and a check digit, for a total of 10 digits. For example, ISBN 0-387-95045-1 has country code 0, publisher code 387, book number 95045, and check digit 1. The check digit is computed by multiplying the leftmost digit by 10, the next digit by 9, and so on, up to the ninth digit from the left, which is multiplied by 2. The products are then added, and the check digit is determined as the smallest integer that when added to the sum will make it a multiple of 11. The check digit is therefore in the range [0,10]. If it happens to be 10, it is replaced by the Roman numeral X in order to make it a single symbol.

8. It is possible to assign 3-digit codes to the letters of the alphabet as shown in Table below. Once this is done, each letter can be converted into three words of cover text according to its 3-digit code. A code digit of 1 is converted into a word with 1, 4, or 7 syllables, a code digit of 2 is converted into a word with 2 or 5 syllables, and a code digit of 3 is converted into a word with 3 or 6 syllables. This way, there is a large selection of words for each word in the cover text.

|   |     |   |     |   |     |   |     |   |     |   |     |   |     |   |     |   |     |
|---|-----|---|-----|---|-----|---|-----|---|-----|---|-----|---|-----|---|-----|---|-----|
| A | 111 | D | 121 | G | 131 | J | 211 | M | 221 | P | 231 | S | 311 | V | 321 | Y | 331 |
| B | 112 | E | 122 | H | 132 | K | 212 | N | 222 | Q | 232 | T | 312 | W | 322 | Z | 332 |
| C | 113 | F | 123 | I | 133 | L | 213 | O | 223 | R | 233 | U | 313 | X | 323 |   |     |

9. Similarly, each of the 26 letters can be assigned a 5-bit code (there are 32 such codes, so six more symbols can be added to the 26 letters and coded). Thus, any message to be hidden becomes a binary string. To hide the string, select some innocuous text and hide each bit in a letter of this text. A bit of zero, for example, may be hidden by changing the letter to lowercase or to italics. Thus, the bits 011000011 may be hidden in the text to be or not by changing it to to Be or nOT. This type of information hiding is sometimes referred to as Bacon's biliteral cipher.

10. Select a keyword, such as blah. Find the serial numbers, 2, 12, 1, and 8 of its letters. Construct a grid with 12 columns and several rows, and mark columns 1, 2, 8, and 12 of each row. The letters of the message are inserted in those positions, four letters per row, and are hidden by filling the grid up with other letters, trying to come up with meaningful text.

11. Assign a letter to each of 26 musical notes, then write your message in the form of musical notes and hope that no one will try to play the music. This idea is due to Gaspari Schotti who published it in his *Schola Steganographica* [Schotti 65].

12. Write or print the data on paper, then photograph it and shrink it to the size of a dot, like the one at the end of this sentence. Send an innocuous letter to the receiver and paste the dot at the end of one of the sentences. The receiver can identify the microdot by holding the letter to the light, looking for a shiny dot.

## SIMPLE DIGITAL METHODS

Modern steganography methods are more sophisticated than the ones presented so far and are based on the use of computers and on the binary nature of computer data. They can be classified into naive steganography (methods where no key is used), secret steganography (methods where a secret key is used), and public-key steganography (methods where an asymmetric key is used, similar to public key in cryptography). All three classes of steganographic methods are based on the fact that every communications process is accompanied by some natural randomness. A steganographic method replaces this randomness by the hidden data without changing the nature of the process. The hidden data end up being *embedded* in the process. A modern personal computer may have tens of thousands of files on one hard disk. Many files are part of the operating system, and are unfamiliar to the computer owner or even to an expert user. Each time a large application is installed on the computer, it may install several system files, such as libraries, extensions, and preference files. A data file may therefore be hidden by making it look like a library or a system extension. Its name may be changed to something like MsProLibMod.DLL and its icon modified.

When placed in a folder with hundreds of similar-looking files, it may be hard to identify as something special. Camouflage is the name of a steganography method that hides a data file *D* in a cover file *A* by scrambling *D*, then appending it to *A*. The original file *A* can be of any type. The camouflaged *A* looks and behaves like a normal file, and can be stored or emailed without attracting attention. Camouflage is not very safe, since the large size of *A* may raise suspicion. When files are written on a disk (floppy, zip, or other types), the operating system modifies the disk directory, which also includes information about the free space on the disk. Special software can write a file on the disk, then reset the directory to its previous state. The file is now hidden in space that's declared free, and only special software can read it. This method is risky, because any data written on the disk may destroy the hidden file.

## DATA HIDING IN TEXT

Text typically has less inherent noise than images, which means that hiding data in text generally results in **low embedding capacity**. Despite this limitation, several creative techniques exist for embedding bits of data in a text file. This section outlines three general approaches, followed by specific algorithms focused on hiding data in text.

**Modifying Spaces:** One method involves hiding data by altering **blank spaces** within the text. A word processor can manipulate:

1. Spaces between words (interword spaces),
2. Spaces at the end of each line, and
3. Spaces after punctuation marks.

Typically, modern word processors automatically manage spacing to justify the right margin and don't allow explicit user control. To enable data hiding, such processors need to be modified to (1) permit user control over space sizes and (2) accurately display space dimensions for retrieving hidden bits.

In basic word processors where spaces are fixed in size, a simple method hides bits at the **end of sentences**:

- **One space** after a period represents a **0**
- **Two spaces** represent a **1**  
Even abbreviations like "Mr." would hide a bit, based on the spacing that follows.

Another method appends **one or two spaces to the end of each line** to encode bits. These trailing spaces are invisible when printed but can be recognized by the word processor. However, they might be removed by other programs during processing.

For **interword spacing**, some processors use extra spaces to justify text alignment. This feature can be exploited to hide data, where:

- **One space** between words = bit **0**
- **Two spaces** = bit **1**

A challenge arises when justification constraints limit spacing (e.g., if only one space is allowed, but a two-space bit is needed). To address such cases, a more refined algorithm uses space patterns around words:

1. **Single space – word – double space** → bit **0**
2. **Double space – word – single space** → bit **1**
3. **Single space – word – single space** → **no data hidden**
4. **Double space – word – double space** → **no data hidden**

This structured interpretation allows for more controlled and less conspicuous data embedding within text.

```
happy_families_are_all_alike_every_unhappy_family_is_unhappy
in_its_own_way_everything_was_in_confusion_in_the_oblonskys
house_the_wife_had_discovered_that_the_husband_was_carrying
on_an_intrigue_with_a_french_girl
```

The **TEX typesetting system** [Knuth 84] offers highly precise control over interword spacing and spacing after punctuation. TEX uses a unit called the **scaled point (sp)**, where one inch equals 72.27 points (pt) and one pt equals 65,536 scaled points—making a single sp extremely small, nearly the size of a visible light wavelength. This means modifying spacing by 1 sp is imperceptible to the human eye. Additionally, TEX can report the exact dimensions of text components, including letters and spaces. Although TEX was originally designed for high-quality book typesetting, these features make it well-suited for **hiding data using space modification techniques**.

**Syntactic methods** use punctuation ambiguities or text modifications that preserve meaning to hide data. Using ambiguous punctuation is easier but riskier, as inconsistencies can be detected, especially by a suspicious observer. On the other hand, preserving meaning while

modifying structure is more secure but challenging to automate, given that computers struggle with semantic understanding. For example, the phrases “*a common, boring, but responsible task*” and “*a common, boring but responsible task*” differ by a comma—this subtle change can encode a bit of data. Although this method has low embedding capacity (one bit per applicable phrase), it is useful in limited contexts. Similarly, abbreviations like “e.g.,” can be tweaked slightly—e.g., writing “*see e.g., page 10a*” versus “*see e.g. page 10a*”—to hide bits.

More sophisticated syntactic approaches involve rephrasing text without altering its meaning. For instance, “*when you finish this, you can go*” versus “*you can go when you finish this*” can be used to hide one bit. Although such methods offer low capacity and often require manual editing, they are robust, as detection typically requires human interpretation. An automated system would find it difficult to identify and interpret such rewordings.

**Semantic methods** embed data based on word choice using tools like a shared **online thesaurus**. The sender and receiver agree on which thesaurus to use. As the decoder scans the cover text, it checks each word’s first synonym list occurrence. If a word, such as *godchild*, does not appear as a synonym for any other word, it’s assumed to carry no hidden data. But if a word like *child* appears as the third entry (e.g., bud, chick, child, kid, minor), it can be used to represent the 2-bit number 2 (binary **01**). In encoding, if the next two bits to be hidden are 01, and *child* fits this position, it can be used. Otherwise, the encoder may replace *child* with another synonym from the list, like *bud* or *chick*, and repeat the process. If none match the target bit pattern, human intervention may be required to find an acceptable alternative.

Another semantic hiding technique uses **sentence-level properties** like the **parity** (odd or even sum) of the ASCII codes of all characters in a sentence to embed data. A word processor adapted for this method would:

1. Accept secret data as a string of bits,
2. Calculate the sentence’s ASCII parity each time a period is typed, and
3. Compare it with the next bit to be hidden.

If the sentence’s parity doesn’t match the target bit, the program alerts the user and prevents further input, forcing the sentence to be rewritten. While such approaches are not yet fully automatable, future systems may support automated generation and adjustment of text for meaningful **semantic data hiding**. For now, **human involvement is essential** in implementing this technique

## INNOCUOUS TEXT

A government agency like the NSA, which intercepts an enormous volume of messages daily, must rely on automated systems to analyze and filter these communications. Computers are first used to detect potentially suspicious messages by evaluating **randomness**. If a message appears highly random, it may indicate encryption and thus be flagged for further scrutiny. However, the randomness could also result from data compression and be entirely innocent. If the message appears to be in plain text, the system proceeds to **linguistic**

**analysis**—first checking **letter and digram frequencies**. If these seem typical, the system then verifies the presence of known words using a **dictionary lookup**, flagging any suspicious terms (like “*bomb*,” “*FBI*,” or “*smuggling*”). If most of the words are valid, the syntax is evaluated. A message that passes all these stages is marked “clean” by the system.

This context introduces a unique steganographic technique: hiding binary data by generating **syntactically correct but semantically meaningless text**. This method creates sentences composed only of valid words in grammatically correct order, fooling standard detection algorithms. The hidden binary data are converted into sequences of words based on a predefined dictionary and syntax model. The process must be **reversible**, allowing the intended receiver to reconstruct the original binary data from the generated text.

Unlike traditional steganography, where secret data is **embedded in a cover medium** (like an image or audio), this method **replaces binary data with text**, making it more akin to a **form of cryptography**. Notable implementations of this approach include **NICETEXT**, developed by Mark Chapman and George Davida, and **Steganosaurus**, by John Walker. NICETEXT uses a **dictionary and grammatical style source** to convert binary data into natural-looking text, which can later be decoded using the same dictionary. Steganosaurus follows a similar concept but does not enforce syntactic correctness, relying only on valid word substitution.

The example dictionary shown here has five grammatical types of words.

```
Article (4): the, a, this, your.
Noun   (16): dog, ball, woman, spring, John, Mary, house, man,
           car, soil, brush, animal, place, sentence, cup, food.
Verb   (16): hit, took, saw, spring, longed, sees, runs, played,
           hates, hear, knew, guesses, liked, walked, believes, work.
Adj.    (8): big, little, blue, green, long, wide, bright, dull.
Prep.   (4): to, in, by, which.
```

A word in the dictionary may have more than one grammatical role, leading to ambiguity. To avoid this, such a word must always occupy the **same position in all its associated types**, and those types must be of **equal length**. For instance, the word *spring*, which functions as both a noun and a verb, appears at position 3 in both respective lists (with positions starting from 0), and both lists contain 16 entries. On the other hand, a word like *long* appears as both a verb and an adjective, but the differing lengths of these types prevent consistent positioning. To resolve this, the verb form *long* is replaced with *longed*. Thanks to this design, the decoder **does not require the original style rules** to retrieve the embedded message.

An alternative approach to handling words with multiple grammatical types is to **merge types**. For example, if *spring* functions as both a noun and a verb, a new dictionary type called *Noun-Verb* can be created, containing *spring* and other similarly dual-purpose words.

The **style source** plays a crucial role by defining **syntax rules** for various sentence structures. A simple sentence format might follow the sequence: *Article, Noun, Verb, Article, Noun*. More complex structures include optional and repeated elements, such as: *Article, (Adj), Noun, Verb, Article, Noun, (Prep, Article, Noun)* where parentheses indicate optional or repeatable components. A more flexible syntax can be

expressed using square brackets to denote a choice between alternatives, for example: *Article, (Adj), Noun, [Verb, Noun-Verb], Article, Noun, (Prep, Article, Noun)*

Consider an example where we encode the 17-bit binary string 01011101001010101. The Nicetext encoder randomly selects a sentence structure from the style source. If the basic syntax rule is selected, the first required word type is *Article*. With four available articles, the encoder uses the first two bits 01 to point to the second article, which is *a*. Next, it processes the following four bits 0111 to choose the 8th noun, *man*. The next four bits 0100 point to the verb *longed*. Then two bits 10 select the article *this*, and the final four bits 1010 point to the noun *brush*. Thus, the encoded sentence becomes “**a man longed this brush.**” After completing one sentence, the encoder can append a period and space, randomly select another syntax rule, and continue encoding the remaining data.

To ensure uniform selection, **dictionary types must contain a number of words that is a power of two ( $2^n$ )**. Each group of input bits serves as an index into a dictionary list of that type. A complication arises when the remaining number of bits is too small to match any style rule. In the earlier example, if only one bit remains (1), and no word list contains just two words, the bit cannot be mapped. One solution is to prepare a structured input string before encoding:

1. A fixed-size binary representation of the data's length
2. The actual binary data
3. A sequence of random bits

For example, for 17 bits of data, the system might generate:  
0000010001                      1010111010010101011                      1100...

Here, the 10-bit prefix 0000010001 encodes the number 17, followed by the 17 bits of data and then random bits. This allows the encoder to continue generating syntactically correct sentences, even after the meaningful data is exhausted. The **decoder begins by reading the fixed-length prefix** to determine how many bits are part of the actual hidden message and then discards the random padding after decoding.

## Suggest another solution.

Creating a large dictionary with word types is a crucial step in text-based steganography. Collecting words is relatively simple, as numerous large-scale word collections are freely available online. For instance, the [FreeBSD Words 01] repository offers a comprehensive list of English words, and the [Gutenberg 01] project provides a massive corpus of literary texts. Writing a program to eliminate duplicate entries is straightforward, but the greater challenge lies in assigning grammatical types—such as noun, verb, adjective, article, or preposition—to each word. There are several strategies to accomplish this. First, manual tagging by an experienced user can be used, though it is best reserved as a final step after attempting automation. Second, one can utilize online dictionaries that already associate types with words. A program can then systematically extract each word and its corresponding type. Third, morphological analysis can be employed to identify root words and derive

various forms—e.g., from the word *compartment*, one can generate *compartmentalize*, *compartmentalization*, *noncompartmentalize*, and more. However, this technique is complex and does not work reliably for all words.

A possible structure for organizing such a dictionary is shown in Figure below . Each grammatical type is assigned a unique number (e.g., 1 for article, 2 for noun, 3 for verb, 4 for adjective, and 5 for preposition), allowing each syntax rule to be represented as a sequence of numbers. These numbers index into an array of structures, one per type, each of which contains the number of words of that type and a pointer to the beginning of that type's word list in the dictionary array. The dictionary itself is a dense character array, where each word is separated by a special terminating ASCII character. A full-fledged implementation of this approach should include tools for building custom dictionaries from various sources, simulating diverse writing styles by example, and potentially using context-free grammars to impose syntactic structure.

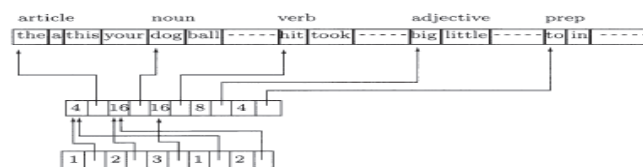


Figure 10.4: Dictionary Organization for Nicetext.

One inherent drawback of this method is data expansion. To illustrate, consider a word length of four bytes; each dictionary type should ideally contain at least 1000 words. Representing an index into such a list requires approximately 10 bits (since  $2^{10} = 1024$ ). Therefore, every 10 bits of original binary data may be replaced by a word and a space, totaling about 5 bytes or 40 bits—resulting in an expansion factor of 4. To mitigate this, compression should be employed. The input data should first be compressed, then encrypted (to improve security), followed by encoding into natural-looking text. The encoded text, though no longer random, can be compressed again before transmission. For example, a 1 MB file might compress to 500 KB. After encryption, the size may grow to 600 KB, and the encoding process could expand this to 2.4 MB. However, compressing the final output may reduce it to around 1.2 MB—only about 20% larger than the original file.

## MIMIC FUNCTIONS

When hiding data in artificially generated text, it is essential that the resulting text can pass automated analysis tools. Such synthetic text must meet at least three core requirements. First, the **letter frequency distribution** should resemble that of natural language. For example, if the artificial text mimics English, then letters like **E** and **T** must appear most frequently, while **Z** and **Q** should remain rare. Second, **most words must be dictionary-valid**. Though occasional presence of proper nouns, jargon, or technical terms may be acceptable, if too many unrecognized words are detected, the text could be flagged as suspicious. Third, the **sentences must conform to grammatical rules**. Any syntactical violations, such as two verbs occurring consecutively, would raise red flags when passed through automatic syntax checkers.

However, simply generating grammatically correct text is not sufficient for steganography. The process must allow **decisions to be made at each step of sentence construction**, where each decision is driven by the next bit or bits of the hidden data. For example, if there are two valid word choices for a position, the next bit (0 or 1) can decide which word to choose. If there are four choices, then two bits can be embedded in that step. In this way, the hidden data actively influence the structure of the generated text.

This concept underpins a technique introduced by **Peter Wayner** [Wayner 92, 02], who developed an algorithm using **context-free grammars (CFGs)** to generate artificial yet natural-looking text that both mimics the statistical characteristics of real language and embeds binary data through selection among grammar production alternatives. His implementation, along with examples and source code, is detailed in [Wayner 02].

A **context-free grammar** consists of a set of rewriting rules—also known as production rules—which may be either explicit or recursive. These rules define how strings can be generated in a language. The full set of strings produced by a CFG forms the **language** defined by that grammar, which can be either finite or infinite, depending on whether recursion is used. The study of formal grammars began in the 1950s with the work of **Noam Chomsky** [Chomsky and Miller, 1958].

A CFG includes four components:

1. A **set of terminal symbols**, which are the words or characters that will appear in the final sentence.
2. A **set of nonterminal symbols**, which act as placeholders for one or more terminals or other nonterminals. These are usually typeset in bold.
3. A **set of productions**, which are rules that describe how a nonterminal can be replaced by a sequence of terminals and/or nonterminals. These rules take the form  $L \rightarrow R$ , where  $L$  is the nonterminal being replaced, and  $R$  is the string of symbols that replaces it.
4. A **start symbol**, which is a designated nonterminal that serves as the initial point of string generation.

The sentence generation process works as follows:

1. Begin with the **start symbol**.
2. Select a production where the start symbol is on the left-hand side and apply it, replacing the start symbol with its corresponding right-hand side.
3. In the resulting text, choose a nonterminal and apply one of its productions.
4. Repeat this process until **only terminal symbols remain**, completing the sentence.

As an example, consider the following simple CFG:

Start  $\rightarrow$  noun verb  
noun  $\rightarrow$  Alice | Bob  
verb  $\rightarrow$  is sending | is receiving

Starting with Start, we replace it with noun verb. Then, replacing noun with *Alice* and verb with *is sending*, we generate the sentence *Alice is sending*. Similarly, depending on the choices made during production, the CFG can also generate *Alice is receiving*, *Bob is sending*, or *Bob is receiving*. These four sentences comprise the entire **language generated by this grammar**.

The grammar is called "context-free" because each nonterminal can be rewritten **independently of its surrounding symbols**—that is, without regard to context. In contrast, **context-sensitive grammars** exist in which production choices depend on surrounding text or conditions.

This CFG-based method of embedding information allows the creation of **natural-looking text** that passes most automated linguistic checks while simultaneously encoding **hidden binary data**.

Find the language generated by the following CFG.

### CFG Definition:

Start → noun verb  
noun → Alice | Bob  
verb → is sending what | is receiving what  
what → type | data | clothes  
type → clean | dirty

### Step-by-Step Derivation:

Generating all possible sentences derivable from this grammar.

#### 1. Expand Start:

Start → noun verb

#### 2. Expand noun:

noun → Alice OR Bob

#### 3. Expand verb:

verb → is sending what OR is receiving what

#### 4. Expand what:

what → type OR data OR clothes

#### 5. Expand type:

type → clean OR dirty

### Now combine all possible expansions:

Let's break into categories based on the two main verb structures.

### Case 1: is sending what

- what → type
  - type → clean
    - Sentence:
      - Alice is sending clean
      - Bob is sending clean
  - type → dirty
    - Sentence:
      - Alice is sending dirty
      - Bob is sending dirty
- what → data
  - Sentence:
    - Alice is sending data
    - Bob is sending data
- what → clothes
  - Sentence:
    - Alice is sending clothes
    - Bob is sending clothes

### Case 2: is receiving what

- what → type
  - type → clean
    - Sentence:
      - Alice is receiving clean
      - Bob is receiving clean
  - type → dirty
    - Sentence:
      - Alice is receiving dirty
      - Bob is receiving dirty
- what → data
  - Sentence:
    - Alice is receiving data
    - Bob is receiving data
- what → clothes
  - Sentence:
    - Alice is receiving clothes
    - Bob is receiving clothes

### Total Sentences Generated:

There are:

- 2 choices for noun (Alice, Bob)
- 2 choices for verb (is sending what, is receiving what)

- 3 choices for what:
  - 2 of them (type) expand to 2 more options → total 2
  - 1 of them (data) → 1 option
  - 1 of them (clothes) → 1 option

So:

- From type:  $2 (\text{noun}) \times 2 (\text{verb}) \times 2 (\text{type} \rightarrow \text{clean/dirty}) = 8$  sentences
- From data:  $2 \times 2 = 4$  sentences
- From clothes:  $2 \times 2 = 4$  sentences

**Total =  $8 + 4 + 4 = 16$  sentences**

### **List of All 16 Sentences in the Language:**

1. Alice is sending clean
2. Alice is sending dirty
3. Alice is sending data
4. Alice is sending clothes
5. Alice is receiving clean
6. Alice is receiving dirty
7. Alice is receiving data
8. Alice is receiving clothes
9. Bob is sending clean
10. Bob is sending dirty
11. Bob is sending data
12. Bob is sending clothes
13. Bob is receiving clean
14. Bob is receiving dirty
15. Bob is receiving data
16. Bob is receiving clothes

### **Conclusion:**

The language generated by this CFG consists of **16 grammatically structured English-like sentences**, combining two subjects (Alice, Bob), two verbs (sending, receiving), and four objects (clean, dirty, data, clothes—derived through nested productions).

## **DATA HIDING IN IMAGES**

Nearly all advanced steganographic techniques conceal a message by embedding it as low-level noise within an image or audio file, which then serves as the **cover file**. This method, however, presents two key limitations. First, the **data hiding capacity** of a cover file is inherently limited, requiring a **large cover file** to embed a significant volume of secret data. Second, once the data is embedded, **lossy compression techniques**—commonly applied to multimedia files—typically **destroy the hidden information**. This restriction suggests that

such images should be compressed using **lossless methods only**. Nevertheless, this chapter demonstrates that it is possible to **embed secret data in images** even when they undergo **lossy compression**.

## LSB ENCODING

Data can be embedded in grayscale or color images due to the human eye's inability to detect minor variations in pixel color values—a fact that is also exploited by lossy image compression techniques [Salomon 00]. The **Least Significant Bit (LSB) encoding** principle hides data bits by embedding them in the least significant bits of image pixels. The result of such embedding is termed a **stegoimage**. The original idea for LSB encoding is commonly attributed to Romana Machado [Stego 01].

For a color image, where each pixel is represented by three bytes (corresponding to red, green, and blue components), each color component can take on 256 different values. The message to be hidden is represented as a binary stream. Each bit of this stream replaces the least significant bit of the next image byte. As both the existing LSB and the hidden bit are binary, the process modifies only about **50% of the LSBs**, translating to **1–2 bits modified per pixel** on average.

However, minor alterations in color may become noticeable if made in regions with uniform color or along sharp boundaries. Hence, an ideal **cover image** should be "busy"—rich in detail and color variations. Enhancing the basic LSB method involves selecting only those pixels that satisfy specific criteria. If the variance of a pixel's neighboring luminance values is either too high (suggesting a boundary) or too low (indicating uniformity), that pixel should be skipped. Moreover, if a modified pixel no longer passes the selection criteria, it must be reverted to its original state.

Comparing the stegoimage with the original image could expose the altered bits, potentially compromising the hidden message. Therefore, the **original image should be destroyed** after embedding. Also, **reuse of a cover image must be avoided**, as multiple stegoimages generated from the same cover may enable an adversary to deduce the embedding method.

If the image is later compressed, it must be compressed using **lossless compression**. Any form of **lossy compression** will likely destroy the embedded data. This makes the method **non-robust**.

Regarding **embedding capacity**, this basic LSB method is limited. One hidden bit per byte yields a data capacity of **12.5% of the cover image's size**. For example, a 1 MB image can embed only 128 KB of hidden data. A popular tool implementing this method is **Stego**, discussed in Section 12.10.1.

To enhance security, numerous **variants using a stego-key** can be applied. Some of these are:

1. **Selective Byte Embedding:** With a key like 1011, hide data in the LSBs of bytes 1, 3, and 4 but not in byte 2; repeat this pattern for subsequent groups of four bytes.
2. **Alternating Bit Position:** Hide bits in chosen bytes, alternating between the LSB and the second LSB.
3. **Pseudo-random Sequence (Basic):** Use the stego-key as a seed to generate a pseudo-random sequence of positive integers  $r_1, r_2, \dots$ . Bits are hidden in pixels at positions  $P_1, P_1 + r_1, P_1 + r_1 + r_2$ , etc. This **reduces capacity** but enhances **security**.
4. **Pseudo-random with Modulo Arithmetic:** As in Variant 3, but when an index exceeds image size, use modulo to wrap around. **Collisions** are avoided by tracking modified pixels.
5. **No Collision Tracking:** Similar to Variant 4, but with no tracking of modified pixels. **Error-correcting codes** should be used to recover corrupted data. The tool **Hide and Seek** follows this method (Section 12.10.2).
6. **Region-based Parity Encoding:** Inspired by BPCS steganography. Divide the image into regions (strips or blocks) using the stego-key. Compute each region's **parity bit**. To embed a bit, modify the LSB of one pixel to match the required parity. The decoder retrieves hidden data by reading parity bits from each region. Selection of the pixel to modify is done carefully to minimize perceptibility—choosing pixels with medium contrast relative to their neighbors.
7. **Complex Pixel Traversal:** No key used. Traverse the image in a complex path like a **space-filling curve** (e.g., Peano or Hilbert curve) instead of a row-wise scan, embedding one bit per visited pixel.
8. **Hash-Based Pixel Addressing** [Aura 96]: Use a **secure hash function (H)** and stego-key  $K$ , divided into  $K_1, K_2, K_3$ . For each data bit  $i$ , compute the target pixel position using:
  9.  $Y = i \div x$
  10.  $X = i \bmod x$
  11.  $Y = (Y + H(K_1 \circ X)) \bmod y$
  12.  $X = (X + H(K_2 \circ Y)) \bmod x$
  13.  $Y = (Y + H(K_3 \circ X)) \bmod y$
  14. Return position =  $Y \times x + X$

Example: For an  $800 \times 600$  image and stego-key 123456789, to embed bit 1001, the output location is  $(X, Y) = (55, 102)$ .

A significant **improvement** involves exploiting human visual sensitivity—people are **least sensitive to blue**, so embedding bits in the **blue channel** of pixels minimizes visual distortion. The trade-off is reduced capacity—from 12.5% to **4.17%**, with one bit per three bytes.

Figures 11.1(a–d) illustrate the visual impact of hiding data in various bit positions of the Lena image. Embedding into more significant bits visibly degrades image quality. Figure 11.2 provides **Matlab code** for embedding text.

Finally, **identifying a stegoimage** may involve analyzing pixel value distributions. Figures 11.1(e–f) demonstrate such distributions before and after embedding, aiding in **steganalysis**.

## Image-Based Steganography Using LSB Encoding – Rephrased Notes

### 1. Principle of LSB Encoding:

Data can be concealed in grayscale or color images by making subtle alterations to pixel colors—changes often undetectable by the human eye. This forms the basis of **LSB (Least Significant Bit) encoding**, a technique where binary data is hidden in the least significant bits of pixel values. The resulting image is called a **stegoimage**.

### 2. Encoding in Color Images:

In a typical color image with 3 bytes per pixel (red, green, blue channels), each color can have 256 shades (8 bits). A binary message is embedded by replacing the LSB of each byte with a bit of the message. Since a bit can only be 0 or 1, about half the LSBs will be changed on average, altering 1–2 bits per pixel.

### 3. Image Selection for Steganography:

To minimize visual distortion:

- **Busy images** (rich in details and colors) are preferred.
- Pixels in **uniform areas** or **sharp edges** should be skipped.
- If a pixel is altered, it must still pass the same visual smoothness tests post-modification.

### 4. Robustness and Security:

- Comparing the original and stegoimage can reveal hidden data. Thus, **the original image should be deleted** after embedding.
- A cover image should **never be reused** for different messages.
- Only **lossless compression** is recommended. Lossy compression can **destroy hidden data**, making basic LSB encoding non-robust.

### 5. Data Hiding Capacity:

Hiding 1 bit per byte means 12.5% of the image size can be used for data (e.g., 128KB hidden in a 1MB image).

## Enhanced Variants Using Stego-Keys

#### 1. Selective Byte Embedding:

- Use a binary key (e.g., 1011) to determine which bytes are used for hiding.

#### 2. Alternate Bit Positions:

- Bits can be hidden not only in LSBs but alternately in the second-least significant bit for added randomness.

#### 3. Pseudo-Random Pixel Selection:

- A key seeds a pseudo-random generator, which selects pixel locations (e.g.,  $P_1, P_1 + r_1, P_1 + r_1 + r_2, \dots$ ).

#### 4. Modulo Indexing with Collision Check:

- When indices exceed the image size, wrap them using modulo. Maintain a list of modified pixels to avoid overwrite.
- 5. **Without Collision List (With Error Correction):**
  - Allows bit overwrite; message integrity is preserved via **error-correcting codes**.
- 6. **Region-Based Parity Encoding (BPCS-inspired):**
  - Divide image into regions.
  - Compute LSB parity for each region.
  - Flip a bit in one pixel to match desired parity.
  - Choose the pixel that least affects image smoothness.
- 7. **Non-linear Pixel Scanning:**
  - Use complex paths (e.g., **Peano** or **Hilbert curves**) to embed data sequentially without a stego-key.
- 8. **Secure Hash-Based Addressing ([Aura 96]):**
  - Use a **secure hash function** with a stego-key to generate pixel positions for embedding.
  - Ensures uniqueness and unpredictability.

### Eye Sensitivity and Improved Embedding

- The human eye is **most sensitive to green** and **least to blue**.
- For reduced perceptibility, embed data in **blue channel** only. This lowers capacity to 4.17% (1 bit per 3 bytes).

### Detection via Laplace Filtering

- **Transform pixels** using a Laplace filter:

$$P[x,y] = P[x-1,y] + P[x+1,y] + P[x,y-1] + P[x,y+1] - 4 * P[x,y]$$

$$P[x,y] = P[x-1,y] + P[x+1,y] + P[x,y-1] + P[x,y+1] - 4 * P[x,y]$$

- Stegoimages may show **anomalous pixel distributions** compared to natural images.
- **Laplace transform** amplifies such anomalies, making it a useful detection tool.

### Redundancy for Robustness

- To resist distortions (e.g., compression, cropping), **embed the same bit multiple times**.
- Increases decoding success rate even after image tampering.
- The **Patchwork technique** is an example of using redundancy effectively.

### A COLOR LOOKUP TABLE

A modern computer displays an image on a screen by means of pixels. The process starts when a program constructs the image in a buffer, termed the *bitmap*, in memory.

Each pixel of the image has an entry in the bitmap, indicating its color. The size of Here is a **rephrased summary of the given content on bitmap-based steganography and luminancechrominance color spaces**, suitable for notes:

## **Bitmap and Color Table-Based Image Representation & Steganography – Rephrased Notes**

### **1. Image Display Using Bitmaps:**

- A modern computer displays images by constructing a **bitmap** in memory—a grid of **pixels**, each storing color information.
- The color depth of each pixel (i.e., how many bits represent a color) varies from **1 bit (2 colors)** to **24 bits (~16.8 million colors)**.
- For example, a **1K × 1K image with 24-bit color** (i.e.,  $1,024 \times 1,024 \times 3$  bytes) results in a **bitmap size of ~3 MB**.

### **2. Color Lookup Table (CLUT) Representation:**

- Most images don't use all 16 million colors. Often, only a few hundred or thousand unique colors appear.
- To save space, images use a **color lookup table (CLUT)**, also known as a **palette**:
  - Example: A  $256 \times 3$  CLUT maps 256 colors using 3 bytes each (RGB).
  - The bitmap stores **indexes (1 byte)** pointing to entries in the table.
- This format is used in image file types like **GIF** and **BMP**.

## **Steganography in Indexed Images**

### **1. Hiding in the Color Lookup Table:**

- **Simple Method:** Modify the **least significant bits (LSBs)** of RGB values in the CLUT.
  - Total capacity:  $256 \text{ entries} \times 3 \text{ bytes} = \mathbf{768 \text{ bits (96 bytes)}}$ .
- **Advanced Method:** Rearranging entries in the CLUT:
  - Swap entries  $i$  and  $j$  and update all bitmap references accordingly.
  - Rearrangement is used to embed data in the LSBs of palette indices.
  - This can hide up to **~200 bits**, but the data can be easily destroyed by randomizing the table.

### **2. Hiding in the Bitmap:**

- The bitmap stores **pointers (indexes)**, not colors.
- Changing an index may drastically change the displayed color (e.g., index 4 vs. 5).
- **Solution:** Sort the CLUT such that adjacent entries are similar in color (using **Euclidean distance** of RGB values).
- Even better: Sort by **luminance** instead of RGB for minimal visual distortion.

### 3. Luminance-Aware Encoding:

- If sorting isn't allowed, advanced hiding can still be done:
  - If a 0 bit needs to be hidden in a **bitmap entry with odd index P**, search for a **similar even-indexed** color Q in CLUT.
  - Replace P with Q in the bitmap to hide the bit **while preserving visual similarity**.

## Color Spaces and Luminance

### 1. RGB vs. Luminance-Based Color Spaces:

- RGB represents colors using **Red, Green, Blue** intensities (0–255).
- However, **human vision is more sensitive to brightness (luminance)** than to exact colors (chrominance).
- Hence, color spaces like **YCbCr** are more perceptually efficient.

### 2. What is Luminance?

- **Luminance (Y)** reflects how bright an area appears to human eyes.
- It's a weighted sum of RGB, assigning:
  - **High weight to green,**
  - **Medium to red,**
  - **Low to blue** (as the eye is least sensitive to it).

### 3. YCbCr Color Space:

- **Y:** Luminance (brightness)
- **Cb:** Blue chrominance ( $B - Y$ )
- **Cr:** Red chrominance ( $R - Y$ )
- Used in video compression and image processing due to its human-vision-friendly representation.

### 4. YCbCr Value Ranges (Digital):

- Y: 16–235
- Cb and Cr: 16–240 (128 when RGB is 0)

### 5. RGB ↔ YCbCr Conversion Equations:

#### RGB to YCbCr:

$$Y = (77/256) * R + (150/256) * G + (29/256) * B$$

$$Cb = -(44/256) * R - (87/256) * G + (131/256) * B + 128$$

$$Cr = (131/256) * R - (110/256) * G - (21/256) * B + 128$$

#### YCbCr to RGB:

$$R = Y + 1.371 * (Cr - 128)$$

$$G = Y - 0.698 * (Cr - 128) - 0.336 * (Cb - 128)$$

$$B = Y + 1.732 * (Cb - 128)$$

## BPCS STEGANOGRAPHY

### 1. Introduction

- **BPCS steganography** was introduced by **Eiji Kawaguchi** in 1997.
- It is known for its **high data-hiding capacity**, typically allowing **50–70%** of the cover image size to be used for embedding.
- Importantly, it **does not increase the file size** of the image.

### 2. Key Concepts

- BPCS works by decomposing the image into **bitplanes** (separate binary layers for each bit of the pixel values).
- In a grayscale image (8 bits/pixel), there are 8 bitplanes; in a color image (24 bits/pixel), there are 24 (8 per color component).
- **Higher-order bitplanes** (like bitplane 8) resemble the original image, while **lower-order bitplanes** (like bitplane 1) appear more **random**.
- Some areas of each bitplane still appear random—these are used for embedding secret data.

### 3. Gray Code Transformation

- Pixel values are converted from binary to **Gray code** before embedding.
- This transformation distributes the **randomness** more evenly across all bitplanes, enhancing data hiding opportunities.

### 4. Embedding Process

- The image is divided into **8×8 bit blocks** within each bitplane.
- Each block is evaluated for **randomness** using a **complexity measure  $\alpha$** :
  - $\alpha = (\text{number of differing adjacent bit pairs}) / (\text{maximum possible number})$ .
  - For an 8×8 block, maximum =  $2 \times 8 \times (8-1) = 112$ .
- A block is considered **random** if  $\alpha \geq \text{threshold}$  (typically **0.3**).
- **Only random blocks** are replaced with 64 bits of hidden data.

### 5. Complexity Threshold

- The  $\alpha$  threshold value ( $\approx 0.3$ ) provides a good balance between hiding capacity and visual quality.
- It is derived from a **Gaussian distribution** of  $\alpha$  values, with:
  - Mean = 0.5
  - Standard deviation ( $\sigma$ ) = 0.047
  - Threshold =  $0.5 - 4\sigma = 0.312 \approx 0.3$

### 6. Conjugation for Non-Random Data Blocks

- Hidden data must appear **random** to blend into random blocks.
- If a compressed 64-bit block doesn't meet the randomness test:

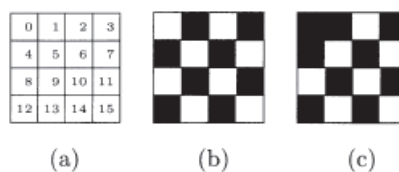
- It undergoes **conjugation**: XOR with a **checkerboard pattern** (starting with white in the top-left).
- This transforms  $\alpha$  to  $1 - \alpha$ , increasing randomness.
- A **conjugation map** records which blocks were conjugated.

## 7. Handling the Conjugation Map

- To reduce map size:
  - If a data block and its conjugate both have  $\alpha$  within [threshold, 1–threshold], no bit is stored in the map.
- The map is itself embedded in the image using the **same BPCS technique**.
- **Avoiding infinite recursion**:
  - Each 8×8 block of the map uses **63 bits for data**, and **1 bit as a flag**:
    - Flag = 0 before embedding.
    - If the block is conjugated, the flag flips to 1.
    - During decoding, the flag reveals whether the block was conjugated.

## 8. Advantages of BPCS

- **High capacity** (up to 70% of image size).
- **No visible distortion** if proper thresholds are used.
- **No need to track embedded locations** — randomness evaluation ensures detectability at decoding.
- **Smart handling of metadata (conjugation map)** avoids increasing image size.



The figure above shows BPCS Steganography: Complexity Threshold and Conjugation Map

### Understanding Complexity Measure ( $\alpha$ )

- In **Bit-Plane Complexity Segmentation (BPCS)**, the **complexity of a block**, denoted as  $\alpha$ , helps determine whether a block is *random-looking*.
- A typical formula for  $\alpha$  in an 8×8 binary block:

$$\alpha = \frac{\text{Number of transitions (horizontal + vertical)}}{2 \times \text{height} \times \text{width}} \quad \alpha = \frac{\text{Number of transitions (horizontal + vertical)}}{2 \times \text{height} \times \text{width}}$$

- **Example (Figure 11.7c):**

$$\alpha = \frac{(3+4+4) + (3+4+4)}{2 \times 4 \times 3} = \frac{22}{24} = 0.917 \quad \alpha = \frac{(3+4+4) + (3+4+4)}{2 \times 4 \times 3} = \frac{22}{24} = 0.917$$

### Threshold for Randomness

- A block is considered "**random enough**" (suitable for embedding secret data) if:

$$\alpha \geq \text{threshold}$$

- Based on experiments, a good threshold value is:

$$\text{threshold} \approx 0.3$$

- This is derived from:
  - Mean  $\alpha$  across many  $8 \times 8$  blocks  $\approx 0.5$
  - Standard deviation ( $\sigma$ )  $\approx 0.047$
  - Hence:

$$\text{threshold} = 0.5 - 4\sigma = 0.5 - (4 \times 0.047) = 0.312 \approx 0.3$$

### Why 0.3?

- Provides high embedding capacity.
- Maintains acceptable visual quality of the stego-image.

### Optimizing the Conjugation Map

- To prevent low-complexity blocks from being detected:
  - A **conjugation operation** (bit inversion using a checkerboard pattern) is applied to convert them into high-complexity ones.
- A **conjugation map** is used to record which blocks were conjugated.

#### Basic Approach:

- One **bit per data block** in the conjugation map to indicate conjugation (1) or not (0).

#### Optimized Approach:

- If both the original data block **and** its conjugate have  $\alpha$  within the **interval**:

$$[\text{threshold}, 1 - \text{threshold}]$$

then:

- The block is *sufficiently random* either way.
- No conjugation is needed.
- **No map bit is required** for such blocks.

**Benefit:** Significantly reduces the **size of the conjugation map**, which increases overall data embedding efficiency.

## GRAY CODES

### 1. Binary vs. Gray Code Representation

- In **standard binary**, the **value of each bit** depends on its **position**. For instance, the binary number 1101 equals  $2^3 + 2^2 + 2^0 = 13$ .
- While this system is computationally efficient, it's **not ideal for image representation**, especially in **bitplane analysis**, because adjacent numbers differ in **multiple bits**.

## 2. Bitplanes in Grayscale Images

- In a grayscale image using **4-bit pixels**, each pixel can have **16 intensity levels** (from 0 to 15).
- If we break such an image into **bitplanes** (b3, b2, b1, b0), each bitplane consists of **one specific bit** from every pixel.
- **Observation:**
  - **Higher-order bitplanes** (like b3) resemble a **binary black-and-white version** of the original image.
  - **Lower-order bitplanes** (like b0) appear **random**.
  - This is because of **pixel correlation** in real-world images: **neighboring pixels are usually similar**, especially in smooth regions.

## 3. Why Standard Binary Bitplanes Vary So Much

- In natural images, **consecutive pixel values** differ slightly.
- But in **binary**, even small value changes often cause **large bit differences**, particularly in the **lower-order bits**.
- Example:
  - Write values 0–15 in binary, and compare their bitplanes:
    - Most significant bits (leftmost) change slowly.
    - Least significant bits (rightmost) flip frequently, appearing random.

## 4. Introduction to Gray Codes

- **Gray code** (or **Reflected Gray Code – RGC**) is an alternative binary representation where **consecutive values differ by only one bit**.
- This feature makes **bitplanes smoother** and **more uniformly random**, which is particularly useful for **image processing** and **steganography** (like BPCS).
- In images using Gray code, adjacent pixels have more **similar Gray code representations**, and hence **bitplane transitions** are **less abrupt**.

## 5. Constructing Gray Codes

- **Recursive Construction Rule:**
  - Start with base: 0, 1
  - Duplicate the list:
    - Prefix the **first copy** with 0.
    - Prefix the **second copy** (reversed) with 1.
  - Example for 2-bit Gray codes:
    - (00, 01) and (11, 10) → Combined: **(00, 01, 11, 10)**
  - Repeat for 3-bit Gray codes:
    - (000, 001, 011, 010) and (110, 111, 101, 100) → Combined: **(000, 001, 011, 010, 110, 111, 101, 100)**

## 6. Bit Change Behavior

- In standard binary:
  - Least significant bit (b0) changes **every number**.
  - b1 changes **every 2 numbers**, b2 every 4, and so on.
- In Gray codes:
  - **Only one bit changes** between any two consecutive values.
  - This makes transitions in bitplanes **smoother**, reducing randomness in lower bits.
- The **first and last** numbers in a Gray code sequence also differ by only one bit, making it **cyclically stable**.

## 7. Application in Image Processing

- Converting image pixel values to **Gray codes** before bitplane segmentation results in:
  - More **balanced distribution** of randomness.
  - Easier **identification of complex or random regions**.
  - **Better performance** in steganography techniques (like BPCS), where data is hidden in visually noisy (random) regions.

| <u>43210</u> | <u>Gray</u> | <u>43210</u> | <u>Gray</u> | <u>43210</u> | <u>Gray</u> | <u>43210</u> | <u>Gray</u> |
|--------------|-------------|--------------|-------------|--------------|-------------|--------------|-------------|
| 00000        | 00000       | <b>01000</b> | 10010       | <b>10000</b> | 00011       | <b>11000</b> | 10001       |
| 00001        | 00100       | 01001        | 10110       | 10001        | 00111       | 11001        | 10101       |
| <b>00010</b> | 01100       | 01010        | 11110       | <b>10010</b> | 01111       | <b>11010</b> | 11101       |
| 00011        | 01000       | 01011        | 11010       | 10011        | 01011       | 11011        | 11001       |
| <b>00100</b> | 11000       | <b>01100</b> | 01010       | <b>10100</b> | 11011       | <b>11100</b> | 01001       |
| 00101        | 11100       | 01101        | 01110       | 10101        | 11111       | 11101        | 01101       |
| <b>00110</b> | 10100       | <b>01110</b> | 00110       | <b>10110</b> | 10111       | <b>11110</b> | 00101       |
| 00111        | 10000       | 01111        | 00010       | 10111        | 10011       | 11111        | 00001       |

```
function b=rgc(a,i)
[r,c]=size(a);
b=[zeros(r,1),a; ones(r,1),flipud(a)];
if i>1, b=rgc(b,i-1); end;
```

The figure above shows how individual bits change when moving through the binary codes of the first 32 integers. The 5-bit binary codes of these integers are listed in the odd-numbered columns of the table, with the bits of integer  $i$  that differ from those of  $i-1$  shown in boldface. It is easy to see that the least significant bit (bit  $b_0$ ) changes all the time, bit  $b_1$  changes for every other number, and, in general, bit  $b_k$  changes every  $k$  integers. The even-numbered columns list one of the several possible reflected Gray codes for these integers.

## LOSSLESS DATA HIDING

### 1. Motivation for Reversible Steganography

Most steganographic techniques **permanently alter** the cover image, even if the change is **visually imperceptible**. Such methods are called **lossy or irreversible**.

However, in certain sensitive applications, the **original cover image must be recovered exactly** after the hidden data is extracted. Techniques that support this are called:

- **Lossless**
- **Reversible**

- **Invertible**
- **Distortion-free**

## 2. Applications of Lossless Data Hiding

- **Medical Imaging:** Embedding patient data (e.g., name, diagnosis) in X-rays without altering the image, as **every pixel may carry diagnostic importance**.
- **Spy Satellite Imagery:** Watermarking satellite images with metadata (e.g., location, time, examiner's notes) without altering pixels that may hold **critical intelligence**.
- **Digital Artwork:** Artists may watermark their work to prevent theft but require **no pixel distortion**, even if invisible to the human eye.

## 3. General Steps for Reversible Embedding

1. **Identify** the image part to be modified.
2. **Compress** this region to preserve the original content.
3. **Append** the compressed version to the hidden data.
4. **Embed** both into the image.

On decoding:

- The secret data and compressed part are extracted.
- The compressed part is decompressed and used to **reconstruct the original image**, restoring it perfectly.

## 4. RS Lossless Data Embedding Method

Based on the work of **Fridrich et al. (2002)**.

### 4.1 Overview

- Start with a cover image  $V$ .
- Choose a region or feature  $B$  of the image to embed data.
- $B$  must be:
  1. **Compressible** (not random).
  2. **Modifiable** without degrading visual quality.

### 4.2 Image Partitioning

- Image of size  $M \times N$  is divided into groups  $G$  of  $n$  adjacent pixels.
- Examples: rows, columns, or blocks of  $n$  pixels.
- Use a **discrimination function**  $f(G)$  to measure the uniformity/smoothness of each group:

$$f(G) = \sum_{i=1}^{n-1} |P_{i+1} - P_i| \quad f(G) = \sum_{i=1}^{n-1} |P_{i+1} - P_i|$$

### 4.3 Flipping Permutations

- Define a **flipping permutation**  $F$ , an **involutionary transformation** ( $F = F^{-1}$ ), e.g., flipping LSBs:
  - $0 \leftrightarrow 1, 2 \leftrightarrow 3, \dots, 254 \leftrightarrow 255$

- **Amplitude A** of F: Average change when applying F to all pixel values in set P:

$$A = \frac{1}{|P|} \sum_{v \in P} |v - F(v)| \quad A = \frac{1}{|P|} \sum_{v \in P} |v - F(v)|$$

#### 4.4 Classifying Pixel Groups

Using  $f(G)$  and  $f(F(G))$ , classify each group into:

- **R (Regular)**:  $f(F(G)) > f(G)$
- **S (Singular)**:  $f(F(G)) < f(G)$
- **U (Unusable)**:  $f(F(G)) = f(G)$

Due to pixel correlation, most groups in natural images fall under **R**.

#### 4.5 Data Embedding Process

- For each group:
  - **Append 0** to the RS string if the group is of type R.
  - **Append 1** if it's type S.
  - Skip type U groups.
- The resulting **RS bitstring** (with mostly 0s) is highly compressible.
- Append the **secret data** to the compressed RS string.
- Embed this combined data back into the image using **flips**:
  - Type-R group represents 1; leave it unchanged.
  - If a bit 0 needs embedding in an R-group → **flip** it to S.
  - Similarly, flip S-groups to R if needed.

#### 4.6 Data Extraction and Restoration

- Scan the stegoimage and rebuild the bitstream using  $R \rightarrow 1, S \rightarrow 0$ .
- Split the bitstream into:
  - Compressed RS string
  - Secret data
- **Decompress** the RS string and **restore** flipped groups to original by referencing RS during a second pass.

#### 4.7 Capacity and Efficiency

- Total groups =  $M \times N / n$
- Usable groups =  $NR + NS$
- Payload (in bits):

$$D = NR + NS - |RS| \quad D = NR + NS - |RS|$$

- The more uneven the R:S ratio, the better the compression and higher the payload.

#### Experimental Observations

- With  $n = 4$  pixels/group:
  - Payload varies from **0.019 to 0.12 bits/pixel**.
  - Higher amplitude flips (e.g., flipping 2nd LSB) yield greater capacity.

## 5. Another Lossless Embedding Method (Pixel Value Substitution)

### 5.1 Steps

1. Identify two similar pixel values  $x$  and  $y$ .
2. Construct sets:
  - $S(x)$ : All pixels with value  $x$
  - $S(y)$ : All pixels with value  $y$
3. Build a binary string  $Z$ :
  - 0 for each  $x$  pixel
  - 1 for each  $y$  pixel
4. Compress  $Z$  and **concatenate** with the secret data to form string  $W$ .
5. Embed  $W$  by substituting:
  - A 0 bit in  $W \rightarrow$  store as pixel  $x$
  - A 1 bit  $\rightarrow$  store as pixel  $y$  (or flip  $x \leftrightarrow y$  accordingly)

### 5.2 Decoding

- Scan the image and rebuild string  $W$  using the  $x/y$  pixel locations.
- Split  $W$  to extract:
  - Compressed  $Z$
  - Secret data
- Decompress  $Z$  and use it to **revert**  $y$  pixels changed during embedding.

### 5.3 Notes

- Effective for **palette images** (e.g., **GIF format**).
- Works well when  $x$  and  $y$  are:
  - **Close in value**
  - Represented frequently but in **uneven proportions** (to enable compression of  $Z$ ).

## LOSSLESS DATA HIDING IN JPEG IMAGES

In lossy steganography, embedding data alters the cover image permanently. However, **lossless (reversible) steganography** allows complete recovery of the original JPEG cover image after the embedded data (payload) is extracted.

Two lossless methods for JPEG images are discussed below. Both operate on the **quantized Discrete Cosine Transform (DCT) coefficients** and restore the original JPEG image without any data loss.

### Prerequisites

- Familiarity with **JPEG compression steps**, especially:
  - Division into  $8 \times 8$  blocks
  - DCT application
  - Quantization of DCT coefficients
  - Huffman encoding

## Method 1: Embedding in LSBs of Selected DCT Coefficients

### Key Idea

Embed data in the **LSBs** of selected quantized DCT coefficients across all 8×8 blocks, such that the original image can be fully recovered.

### Steps – Encoder

1. **Block Selection:** Divide the JPEG image into 8×8 blocks. Choose fixed positions (i, j) (e.g., medium-frequency) across all blocks.
2. **Coefficient Extraction:** For each block k, extract the DCT coefficient  $d_k(i, j)$  and compute the **quantized value**:

$$D_k(i, j) = \text{round}\left(\frac{d_k(i, j)}{Q(i, j)}\right) \quad d_k(i, j) = \text{round}(Q(i, j)D_k(i, j))$$

3. **String Construction:** Gather the **LSBs** of all selected  $D_k(i, j)$  values to form a binary string B. To handle negative values correctly:

$$\text{LSB}(D_k) = 1 - (|D_k| \bmod 2) \quad (\text{for negative values}) \quad \text{LSB}(D_k) = 1 - (|D_k| \bmod 2) \quad \text{quad} \quad \text{LSB}(D_k) = 1 - (|D_k| \bmod 2) \quad (\text{for negative values})$$

4. **Compression:** Compress string B to C, using entropy coding.
5. **Payload Formation:** Append secret data to compressed C → form the payload.
6. **Data Embedding:** Embed the payload into the LSBs of the selected coefficients. Store payload length as part of the data.
7. **JPEG Recompression:** Recompress using standard JPEG Huffman coding to produce the **stegoimage**.

### Position Selection Tips

- **Avoid very low (0–1)** and **very high (6–7)** frequencies (which introduce visual artifacts or poor compressibility).
- Prefer **medium-frequency coefficients** and **chrominance channels** (less noticeable to the human eye).

### Steps – Decoder

1. **Input JPEG:** Decompress the stegoimage to retrieve DCT coefficients.
2. **Recreate Sequence S:** Use the same (i, j) pairs and block order.
3. **Extract LSBs:** Visit selected coefficients using the pseudo-random generator, extract LSBs to form string C.
4. **Decompression:** Decompress C to get the original LSBs string B.
5. **Restore Coefficients:** Use B to restore original LSBs (including handling of negative values).
6. **Recompress:** Replace modified coefficients and recompress the image, producing the **original JPEG**.

## Method 2: Embedding via Quantization Table Modification

### Key Idea

Ensure the **LSBs of certain DCT coefficients are 0**, allowing clean embedding of payload bits. This is achieved by manipulating the quantization table.

### Steps – Encoder

1. **Read JPEG:** Decompress to access DCT coefficients and quantization table  $Q(i, j)$ .
2. **Modify Quantization Table:**
  - If  $Q(i, j)$  is **even**:
    - Divide  $Q(i, j)$  by 2.
    - Multiply all  $Dk(i, j)$  divisible by  $Q(i, j)$  by 2  $\rightarrow$  their LSB becomes 0.
  - If  $Q(i, j)$  is **odd**:
    - Floor-divide by 2.
    - Multiply all related  $Dk(i, j)$  by 2.
    - Include the **(i, j)** position as a **flag** in the payload to identify modified quantization values.
3. **Embed Payload:** Embed secret bits in the LSBs (now all 0s) of the adjusted DCT coefficients.
4. **Update Quantization Table:** Include modified table in the JPEG file (JPEG allows custom tables).
5. **Compress to JPEG:** Generate the stegoimage.

### Drawbacks

- Modified quantization tables must be stored in the JPEG.
- May raise **suspicion** in forensic or steganalysis checks.
- **Weaker steganographic security**, though still visually lossless.

### Decoder Steps

1. **Read JPEG Stegoimage:** Extract modified quantization table and DCT coefficients.
2. **Identify Embedding Locations:** Use table or flags to find  $(i, j)$  pairs.
3. **Extract LSBs:** Collect LSBs from the modified coefficients to retrieve secret data.
4. **Restore Quantization Table and Coefficients:** Reverse the changes to reconstruct original image.

### Variations

- Instead of halving  $Q(i, j)$ , set it to 1, and multiply the DCT coefficient by the original  $Q(i, j)$ .
- This avoids quantization distortion but increases **file size** due to the lack of coefficient quantization.

| Feature          | Method 1                                    | Method 2                                |
|------------------|---------------------------------------------|-----------------------------------------|
| Embedding Area   | LSBs of selected quantized DCT coefficients | Coefficients with even/odd quantization |
| Image Distortion | Low (careful position choice)               | Very low (can avoid)                    |

| Feature                        | Method 1                   | Method 2                           |
|--------------------------------|----------------------------|------------------------------------|
|                                |                            | quantization)                      |
| <b>Compression Impact</b>      | Uses standard quantization | Custom table may reduce efficiency |
| <b>Stego Security</b>          | Stronger                   | Weaker (custom quant. table)       |
| <b>Recover Original Image?</b> | Yes                        | Yes                                |
| <b>Custom Tables Used?</b>     | No                         | Yes                                |

## LOSSLESS DATA HIDING IN GIF IMAGES

The **GIF (Graphics Interchange Format)** is ideal for **lossless data embedding** because:

- It uses a **color lookup table (palette)** of up to 256 colors.
- Each image pixel stores only an index (pointer) to this palette.
- The pointer sequence is compressed using **LZW (Lempel-Ziv-Welch)** compression.

### Version 1: When Fewer than 256 Colors Are Used

This version exploits **unused palette entries** to embed secret data **without any visual distortion**.

#### Encoding Steps:

1. **Decompress the image** to get the list of color pointers.
2. **Build a histogram** to find the most frequent color in the image — call it color d.
3. **Create two palette entries** (i and j) both pointing to color d. These represent secret bits 0 and 1 respectively.
4. **Scan the image** again:
  - If the next bit is 0, set the next occurrence of color d to point to entry i.
  - If the bit is 1, point it to entry j.
5. **Compress the modified pointer list** with LZW and save the image.

**Lossless:** No change in actual pixel colors.

**Capacity:** Equal to the number of pixels of color d.

#### Extending Capacity:

- Use **multiple frequently occurring colors** (say, top C colors).
- Each color gets duplicated in the palette and assigned to bit pairs (00, 01, 10, 11, etc.).
- **Embed bit pairs** in corresponding pixels.

#### Metadata Storage:

- To decode correctly, the decoder needs to know how many colors C were used.
- Store C in **palette entry 0**, move the original color to an unused entry, and update the relevant pointers.

### Decoder Steps:

1. **Decompress the GIF image.**
2. **Check palette entry 0** for the number  $C$  of embedding colors.
3. **Rebuild the histogram** to find the top  $C$  colors.
4. **Find their duplicated entries** and decode bits accordingly.
5. Optionally: **Revert all changes** to restore the original palette and pointer list (to erase traces of embedding).

### Version 2: When All 256 Palette Entries Are Used

This version allows embedding even when **no palette entries are free**, by **reusing similar-looking colors** with different pointer sets.

### Encoding Steps:

1. For each palette index  $i$ , identify:
  - $S(i)$  = all pixels that use color  $i$
  - $C_i$  = the color at palette index  $i$
2. **Select two indices  $i$  and  $j$**  where:
  - $C_i$  and  $C_j$  are **visually similar**
  - $S(i)$  and  $S(j)$  are of **very different sizes**
3. **Move  $C_i$  and  $C_j$  to palette entries 0 and 1**, adjusting all pointers accordingly.
4. **Build bitstring  $B$**  by:
  - Appending 0 for each pointer to  $C_i$
  - Appending 1 for each pointer to  $C_j$
5. **Compress  $B$  to  $C$** , then concatenate with the secret data to form the payload.
6. **Embed the payload:**
  - If a pixel points to  $C_i$  and bit = 0, no change.
  - If mismatch, flip pointer from  $i \leftrightarrow j$ .

**Mild distortion may occur**, but it's visually imperceptible because similar colors are used.

### Decoder Steps:

1. **Identify entries 0 and 1** of the palette as  $C_i$  and  $C_j$ .
2. **Scan the image:**
  - Pointer  $i \rightarrow$  bit 0
  - Pointer  $j \rightarrow$  bit 1
3. **Extract the bitstream**, separate compressed string  $C$  and secret data.
4. **Decompress  $C$  to  $B$ .**
5. **Restore the original pointers** using  $B$ .

### Advantages of GIF Steganography

**Perfect reversibility** in most cases.

**Palette-based manipulation** avoids pixel-level noise.

**Leverages LZW compression** to reduce overhead from embedding

## Key Differences Between the Two GIF Methods

| Feature             | Version 1                           | Version 2                                         |
|---------------------|-------------------------------------|---------------------------------------------------|
| Palette entries     | Fewer than 256 colors               | All 256 used                                      |
| Visual distortion   | None                                | Minimal                                           |
| Embedding technique | Duplicate palette entries           | Flip between similar color pointers               |
| Data capacity       | Based on frequency of reused colors | Based on difference in pointer group sizes        |
| Compression method  | LZW                                 | LZW / any efficient method                        |
| Restoration         | Exact, since original colors remain | Nearly exact; minor color changes if not restored |

## QUESTION BANK

## **I. Data Hiding in Text**

1. Explain the basic features and challenges of data hiding in text. How is it different from image or audio steganography?
2. Discuss various real-world applications of data hiding in text. Highlight both security-related and non-security-related uses.
3. Define digital watermarking. Explain how it is used for copyright protection and data authentication.
4. Describe intuitive methods used for data hiding in text. Illustrate your answer with simple examples.
5. Explain any two simple digital methods for hiding data in text, such as font manipulation, line shifting, or word substitution.
6. What is the concept of 'Innocuous Text'? How does it help in hiding data without raising suspicion?
7. Define mimic functions in text steganography. Explain how they help make stego-text resemble natural language text.
8. Explain formatting-based text steganography techniques. How does it differ from semantic-based approaches?
9. Discuss the whitespace manipulation technique for hiding data. What are its limitations in practical use?
10. Explain the use of structured documents (like XML or HTML) for hiding data in text. Provide examples.
11. Discuss synonym substitution and abbreviation expansion as methods of data hiding in text.
12. What are the linguistic or statistical anomalies introduced in stego text? How can they be used to detect hidden messages?
13. Differentiate between cover text and stego text. What factors must be considered when choosing a suitable cover text?
14. Explain how hiding data in text can have ethical or legal implications. Provide relevant scenarios.
15. Discuss the role of dictionary-based encoding in generating mimic text for data hiding.

## **II. Data Hiding in Images**

16. Explain the Least Significant Bit (LSB) encoding technique in image steganography. Illustrate with an example.
17. Describe the working of BPCS (Bit-Plane Complexity Segmentation) steganography. How does it improve hiding capacity?
18. Compare LSB encoding and BPCS steganography in terms of imperceptibility, robustness, and payload capacity.
19. Discuss the image properties (resolution, contrast, color depth) that influence the choice of image steganography technique.
20. Explain how grayscale and color images differ in LSB encoding and data embedding capacity.
21. How can histogram analysis be used to detect LSB steganography? Illustrate with a simple graph or example.
22. Describe the importance of human visual system (HVS) properties in designing imperceptible image steganography methods.

23. In what type of application scenarios is BPCS steganography more suitable than LSB? Justify your answer.
24. Construct a comparative table of image steganography techniques based on robustness, capacity, and imperceptibility.
25. How do image resolution and pixel depth affect the effectiveness and security of image-based data hiding?
26. What are the common steganalysis techniques used for detecting hidden data in images? Suggest countermeasures.
27. Explain the process of embedding and extracting hidden data in LSB-based image steganography using step-by-step procedure.
28. Evaluate the trade-offs involved in increasing the payload in image steganography while maintaining visual fidelity